

Development Environment Setup — Scoping Document

Reproducible, prebuilt FreeCAD toolchain for autonomous Cloud Agents.

Repository `brucecodes2023/FreeCAD_BT` Package manager `pixi (conda-forge)` Target `linux-64`

SUMMARY

FreeCAD is a large C++ / Python parametric CAD application built on OpenCASCADE, Coin3D, and Qt6. This document scopes the work required to make the repository *fully usable* inside Cursor Cloud Agents: a deterministic base image, an idempotent install phase that compiles the application from source, and an end-to-end validation flow that proves the environment works before it is saved as a reusable prebuilt snapshot.

Objectives

- Provide a one-command, reproducible setup that any future Cloud Agent inherits automatically.
- Reuse the project's canonical, pinned toolchain (`pixi + pixi.lock`) rather than inventing a parallel dependency story.
- Compile FreeCAD once during environment `install` so prebuilt snapshots boot ready-to-run.
- Demonstrate the environment end to end: build, automated tests, and a real modeling action.

Current State

The repository already defines a complete, cross-platform build description. Key findings from the codebase analysis:

- **Dependency management:** `pixi.toml + pixi.lock` pin every dependency from `conda-forge` (OpenCASCADE 7.8, Qt6 6.8, PySide6, Coin/pivy, SMESH, VTK, compilers, `ccache`, `mold`).
- **Build system:** CMake with presets in `CMakePresets.json`; `pixi` tasks wrap config-

ure / build / install / test for each platform.

- **Reference CI:** `.github/workflows/sub_buildPixi.yml` encodes the canonical Linux flow — `configure-release` → `build-release` → `install-release` → `tests`.
- **Submodules:** five git submodules (OndselSolver, GSL, Coin, pivy, AddonManager) fetched via the `initialize` task.
- **No prior Cloud Agent config:** no committed `.cursor/environment.json` existed before this work.

Scope

In scope

- A committed `.cursor/environment.json` plus an idempotent `.cursor/install.sh`.
- Installing `pixi` onto `PATH`, fetching submodules, and materializing the locked environment.
- Configuring, building, and installing FreeCAD with the `release` preset (matching CI).
- Local validation on the VM: full build, C++ / Python self-tests, and a headless modeling action.
- Snapshot, prebuilt-build verification, and a proposed configuration for review.

Out of scope

- Changes to FreeCAD application or build-system source (environment configuration only).
- Non-Linux targets (macOS / Windows) — already covered by the project's own CI matrix.
- Dependency upgrades or `pixi.lock` regeneration.
- Packaging, release artifacts, or deployment.

Technical Approach

Base image

Use Cursor's default Ubuntu image. It already provides `git`, `curl`, and passwordless `sudo`; the full compiler toolchain arrives through `pixi`, so a custom Dockerfile is unnecessary. The `pixi` binary is installed once to `/usr/local/bin` so every agent shell finds it without shell-profile edits.

Lifecycle placement

- `install` — durable, idempotent, source-derived work: install `pixi`, fetch submodules, resolve the locked environment, then `configure / build / install` FreeCAD. `ccache` keeps re-runs incremental.
- `start / terminals` — none required. FreeCAD is a desktop application, not a long-running service; GUI runs happen on demand under a virtual display (`xvfb`).

Prebuilt snapshots

Because the compile happens in `install`, the resulting binaries are baked into the environment snapshot. Fresh agents boot with FreeCAD already built and do not pay the compilation cost again.

Deliverables

- `.cursor/environment.json` — declares the environment and its install entry point.
- `.cursor/install.sh` — idempotent bootstrap + build script.
- This scoping document (source + compiled PDF).
- Validation evidence: build logs, passing tests, and a headless modeling artifact.

Validation Plan

Check	Success criterion
Configure	<code>configure-release</code> resolves all dependencies without error.
Build	<code>build-release</code> compiles every target (8092) cleanly.
Install	<code>install-release</code> stages the runnable application.
CLI self-tests	<code>FreeCADCmd -t 0</code> passes the test suite.
C++ unit tests	<code>ctest</code> green across module test runners.
Modeling action	A scripted part is created, and geometric properties (volume / area) verified.
Idempotence	Re-running <code>install</code> converges with no rebuild churn.
Fresh agent	An agent booted from the prebuilt build repeats the checks.

Risks & Mitigations

Risk	Mitigation
Long initial compile (thousands of TUs)	Runs once in <code>install</code> ; snapshot bakes in binaries; <code>ccache</code> accelerates re-runs.
Headless GUI (no physical display)	Exercise the GUI/tests under <code>xvfb</code> ; core validation via the CLI path.
Network access to conda-forge / GitHub	Uses the pinned <code>pixi.lock</code> ; egress is currently unrestricted for the required hosts.

Handoff

Once local validation and prebuilt-build verification pass, the configuration is proposed for review. Saving it in the Environment panel is the single remaining manual action and makes the tested setup available to all future Cloud Agents.